

Queue using Array - Simple Implementation

Last Updated : 20 Sep, 2025

A queue is a linear data structure that follows the **FIFO (First In, First Out)** principle. The first element inserted is the first one to be removed.

Declaration of Queue Using Array:

A queue can be implemented using an array, and there are two main ways:

1. Infinite (or Dynamically Growable) Array Queue
2. Fixed-Size Array Queue

Infinite (or Dynamically Growable) Array Queue:

We can implement a queue using a conceptually infinite array by maintaining only a **front pointer**. The front pointer tracks the first valid element.

- **Enqueue:** Insert at the next available position at the end. No rear pointer is required.
- **Dequeue:** Remove the element at front and increment the front pointer.

The space before front is never reused, and unlike basic array implementations, we do not shift elements after each dequeue. This ensures both enqueue and dequeue operations run in **O(1)** time with a simple design.

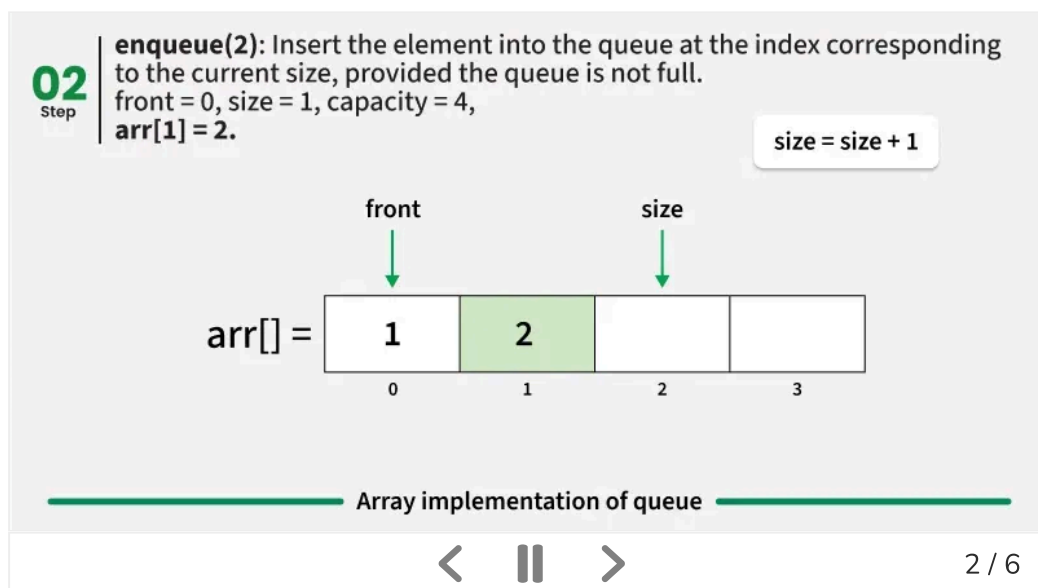
Limitations:

- **Wasted space:** The elements before the front pointer are never reused, so memory can be wasted if many elements are dequeued.
- **Infinite array assumption:** We assume the array is conceptually infinite. In practice, memory is finite, so very large queues can cause memory issues.

Fixed-Size Array Queue

In this article, we will mainly discuss the queue implementation using a **fixed-size array**. In such an array-based queue, we maintain:

- A fixed-size array `arr[]` to store the elements.
- A variable `size` to track the current number of elements in the queue.
- A variable `capacity` to represent the maximum number of elements the queue can hold.



C++

C

Java

Python

C#

JavaScript

```
class myQueue {  
    private int[] arr;  
    private int capacity;  
    private int size;  
  
    // Constructor  
    public myQueue(int capacity) {  
  
        //Maximum number of elements the queue can hold.  
    }  
}
```

```
    this.capacity = capacity;

    // Array to store queue elements.
    arr = new int[capacity];

    // Current number of elements in the queue.
    size = 0;
}
}
```

Operations on Queue

Enqueue (Insert):

- Add element at the end of the queue if space is available; otherwise, it results in an Overflow condition.
- Time: $O(1)$, Space: $O(1)$

[C++](#)[C](#)[Java](#)[Python](#)[C#](#)[JavaScript](#)

```
void enqueue(int x) {
    if (size == capacity) {
        System.out.println("Queue Overflow");
        return;
    }
    arr[size++] = x;
}
```



Dequeue:

- Remove element from the front of the queue; if the queue is empty, it results in an Underflow condition.
- Time: $O(n)$ (because of shifting) , Space : $O(1)$

[C++](#)[C](#)[Java](#)[Python](#)[C#](#)[JavaScript](#)

```
void dequeue() {
    if (size == 0) {
        System.out.println("Queue Underflow");
        return;
    }

    for (int i = 1; i < size; i++) {
        arr[i-1] = arr[i];
    }
    size--;
}
```



getFront (Peek):

- Return first element if not empty, else -1.
- Time: $O(1)$, Space: $O(1)$

C++

C

Java

Python

C#

JavaScript

```
int getFront() {  
    if (size == 0) {  
        System.out.println("Queue is empty");  
        return -1;  
    }  
  
    return arr[0];  
}
```



getRear():

- Return last element if not empty, else -1.
- Time: $O(1)$, Space: $O(1)$

C++

C

Java

Python

C#

JavaScript

```
public int getRear() {  
    if (isEmpty()) {  
        System.out.println("Queue is empty!");  
        return -1;  
    }  
  
    return arr[size - 1];  
}
```



isEmpty():

- Checks whether the queue has any elements or not.
- Returns true if the queue is empty, otherwise false.
- Time: $O(1)$, Space: $O(1)$

C++

C

Java

Python

C#

JavaScript

```
public boolean isEmpty() {  
    return size == 0;  
}
```



isFull():

- Checks whether the queue has reached its maximum capacity.

- Returns true if the queue is full, otherwise false.
- Time: $O(1)$, Space: $O(1)$

C++

C

Java

Python

C#

JavaScript

```
public boolean isFull() {  
    return size == capacity;  
}
```



Full Implementations of Queue using Array

C++

C

Java

Python

C#

JavaScript

```
class myQueue {  
    private int[] arr;  
    private int capacity;  
    private int size;  
  
    // Constructor  
    public myQueue(int capacity) {  
        //Maximum number of elements the queue can hold.  
        this.capacity = capacity;  
        // Array to store queue elements.  
        arr = new int[capacity];  
        // Current number of elements in the queue.  
        size = 0;  
    }  
  
    // Check if queue is empty  
    public boolean isEmpty() {  
        return size == 0;  
    }  
  
    // Check if queue is full  
    public boolean isFull() {  
        return size == capacity;  
    }  
  
    // Enqueue  
    public void enqueue(int x) {  
        if (isFull()) {
```



```
        System.out.println("Queue is full!");
        return;
    }
    arr[size] = x;
    size++;
}

// Dequeue
public void dequeue() {
    if (isEmpty()) {
        System.out.println("Queue is empty!");
        return;
    }
    for (int i = 1; i < size; i++) {
        arr[i - 1] = arr[i];
    }
    size--;
}

// Get front element
public int getFront() {
    if (isEmpty()) {
        System.out.println("Queue is empty!");
        return -1;
    }
    return arr[0];
}

//Get last element
public int getRear() {
    if (isEmpty()) {
        System.out.println("Queue is empty!");
        return -1;
    }
}
```

[Review Questions](#) [Quizzes](#) [Must Do](#) [Advanced DSA](#) [System Design](#) [Aptitude](#)

[Sign In](#)

```
}
```

```
public class Main {
    public static void main(String[] args) {
        myQueue q = new myQueue(3);
```

```
q.enqueue(10);
q.enqueue(20);
q.enqueue(30);
System.out.println("Front: " + q.getFront());

q.dequeue();
System.out.println("Front: " + q.getFront());
System.out.println("Rear: " + q.getRear());

q.enqueue(40);

    }
}
```

Output

Front: 10

Front: 20

Rear: 30

We can notice that the Dequeue operation is $O(n)$ which is not acceptable. The enqueue and dequeue both operations should have $O(1)$ time complexity. That is why if we wish to implement a queue using array (because of array advantages like cache friendliness and random access), we do [circular array implementation of queue](#).

[Comment](#)[More info](#)

Explore

DSA Fundamentals

Data Structures

Algorithms

Advanced

Interview Preparation

Practice Problem



Corporate & Communications Address:

A-143, 7th Floor, Sovereign Corporate Tower, Sector- 136, Noida, Uttar Pradesh (201305)

Registered Address:

K 061, Tower K, Gulshan Vivante Apartment, Sector 137, Noida, Gautam Buddh Nagar, Uttar Pradesh, 201305



Company

About Us
Legal
Privacy Policy
Contact Us
Advertise with us
GFG Corporate Solution
Campus Training Program

Tutorials

Programming Languages
DSA
Web Technology
AI, ML & Data Science
DevOps
CS Core Subjects
Interview Preparation
GATE
Software and Tools

Explore

POTD
Job-A-Thon
Community
Blogs
Nation Skill Up

Courses

IBM Certification
DSA and Placements
Web Development
Programming Languages
DevOps & Cloud
GATE
Trending Technologies

Videos

DSA
Python
Java
C++
Web Development
Data Science
CS Subjects

Preparation Corner

Aptitude
Puzzles
GfG 160
DSA 360
System Design

@GeeksforGeeks, Sanchhaya Education Private Limited, All rights reserved